BENCHMARK · 42 STACKS · 1 SMALL SERVER

How many users can your frontend serve on a $12 server?

Same site, same API, same tiny box. Only the frontend changes: Next.js, Remix, TanStack, SvelteKit, Nuxt, Angular and friends, plus Express and Hono on Node and Bun, Flask, Django, Rails, Laravel, Spring, ASP.NET, Phoenix, Go and Rust.

THE SETUP

Two $12-sized servers and a load generator

k6

Simulated users on separate cores. Thousands of browsing sessions at once.

→

Frontend server

The thing we test. Renders HTML for every page view. **`0.4 vCPU · 2 GB · no swap`**

→

API: Rust + SQLite

50k users, 500k posts, 2M likes. Answers a feed in ~1 ms. **`0.4 vCPU · 2 GB`**

Each server is a Docker container pinned to its own physical core and throttled to 0.4 vCPU, closer to a shared $12 droplet than a full desktop core. Every run starts with a fresh database and a fresh frontend container.

WHAT IS ONE USER?

One user ≈ 0.09 page requests a second

1
Open the feed
20 newest posts, rendered on the server

2
Read for 3-7 s
then open one post

3
Read for 3-8 s
like it 15% of the time, post 2%

4
Idle 5-15 s
then start again

So 1,000 users ≈ 90 page renders a second. A run passes when **p95 < 500 ms, p99 < 1 s and errors < 1%**.

FINDING THE LIMIT

Read the bottleneck, step 10%, hold 5 minutes

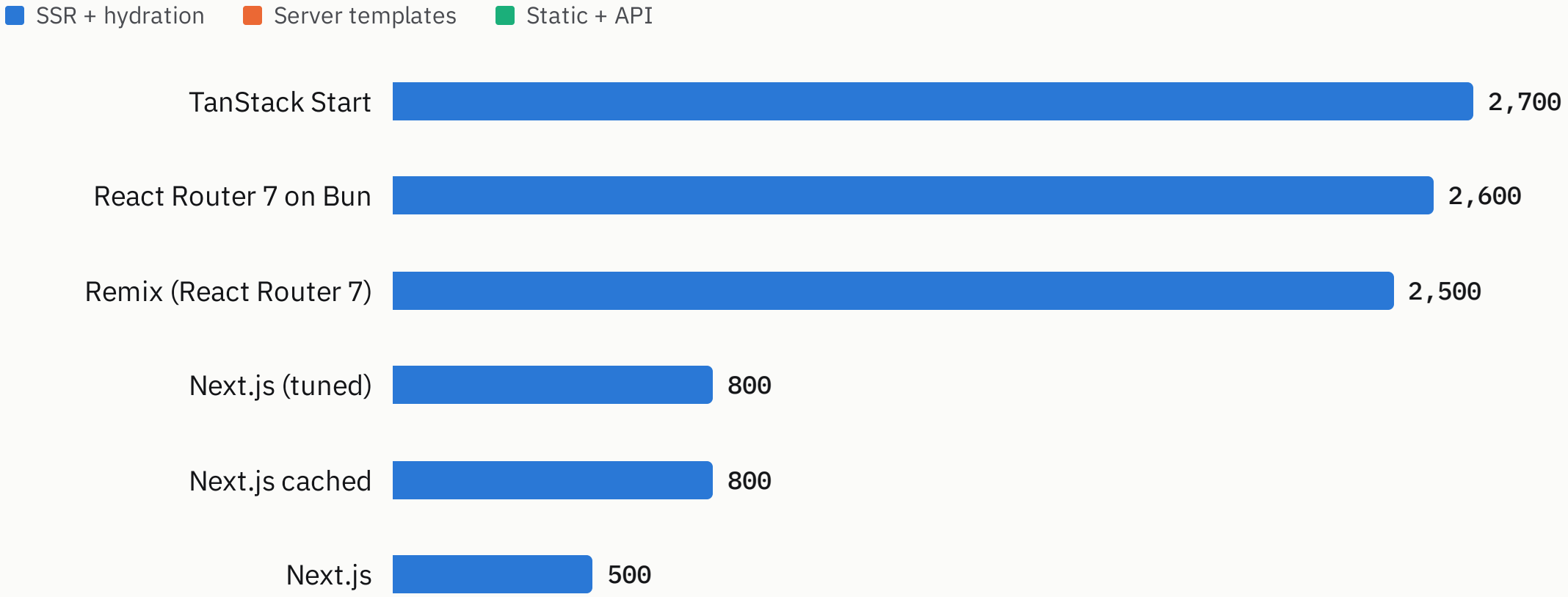
A 2-minute test run shows which CPU is busiest; the search jumps to where it would reach 95%, then moves ±10% per run (bisecting if a jump overshoots). The score is the most users that pass a **5-minute hold**, stepping down 10% from the best 2-minute pass. React Router on Bun, in order:

~~3,100~~	2,900	~~**2,900 · 5 min**~~	**2,600 · 5 min**

Struck through = failed. Bold = a 5-minute hold. 2,900 passed for 2 minutes but not for 5: sustained load is harder. Searches stop at 30,000 users; above that we estimate.

RESULTS · THE REACT FAMILY

Same React, 5× apart



Next.js App Router renders React Server Components to a serialized tree, renders that to HTML, and inlines the tree in the page. Remix and TanStack Start render the same components in one pass.

Five ways to make Next.js serve more

1.6×

Tune itself

tuned	800

config: no staged render, props not trees

1.4×

Runtime

Bun	700
Deno	500

same code on Bun / Deno

24×

Cache

nginx cache	12,100
smart cache	3,600
use cache	800

nginx in front; use cache / ISR inside

5.4×

Same React, other framework

TanStack	2,700
RR7 on Bun	2,600
React Router 7	2,500

one render pass, small data blob

60×+

Other framework / language

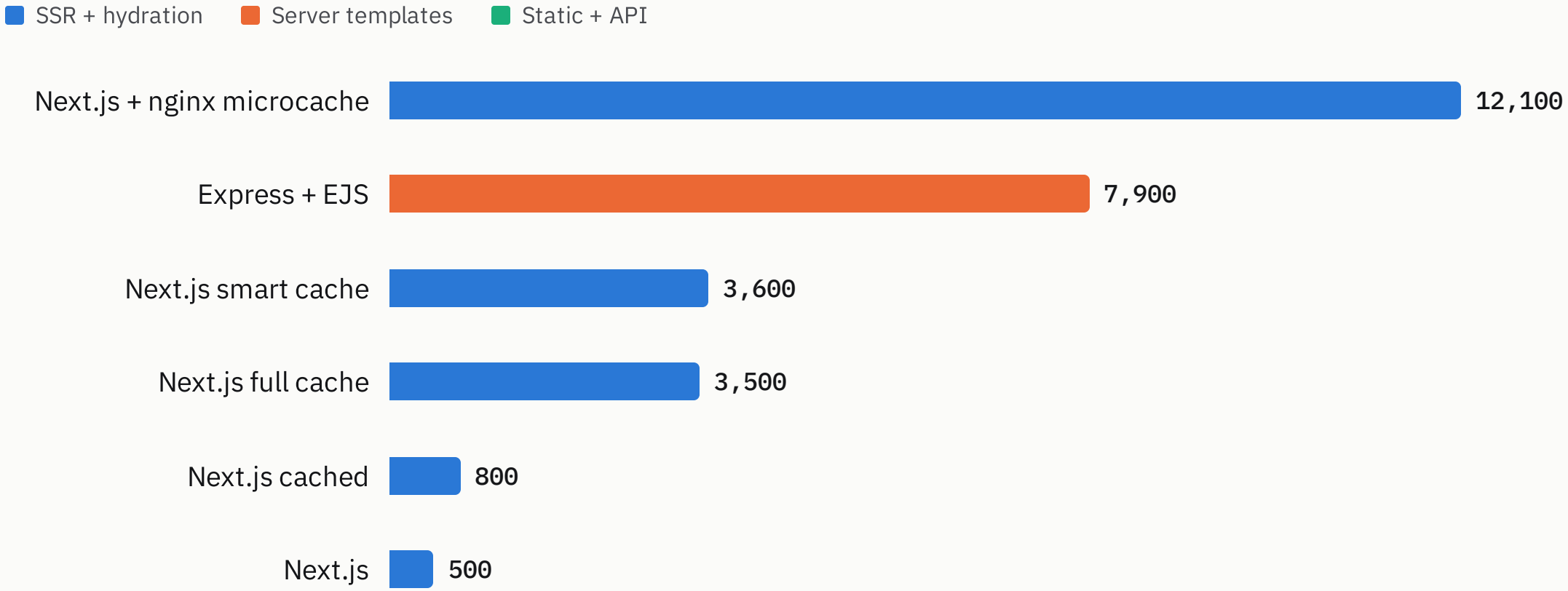
Rust	30,000+
Express, Bun	24,900
Go	19,000
Leptos	15,500
HonoX	16,100
Express, Node	9,700

server templates; Leptos and HonoX hydrate

Tuning Next.js itself (1.6×) beats switching runtime (1.4×). A proxy cache gives 24× for anonymous pages, and the big multiples need a different framework or language.

CACHING NEXT.JS

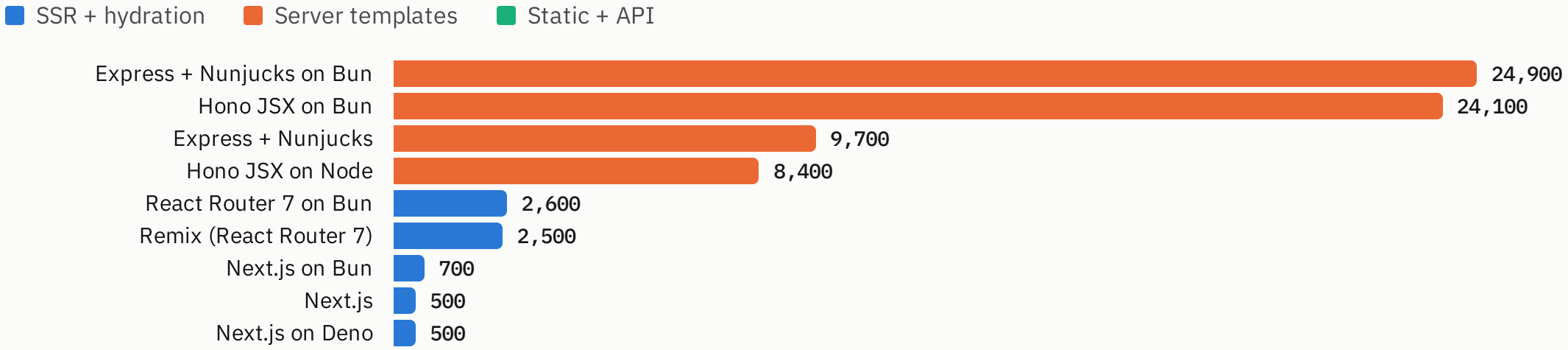
Cache inside Next.js: 7×. In front of it: 24×



Inside Next.js, a stored page is cheap, but new posts, writes through route handlers, 39 KB pages and GC pauses set a ceiling. An nginx microcache (feed 1 s, posts 5 s, anonymous only) answers hits without waking Node at all.

SWITCHING THE RUNTIME

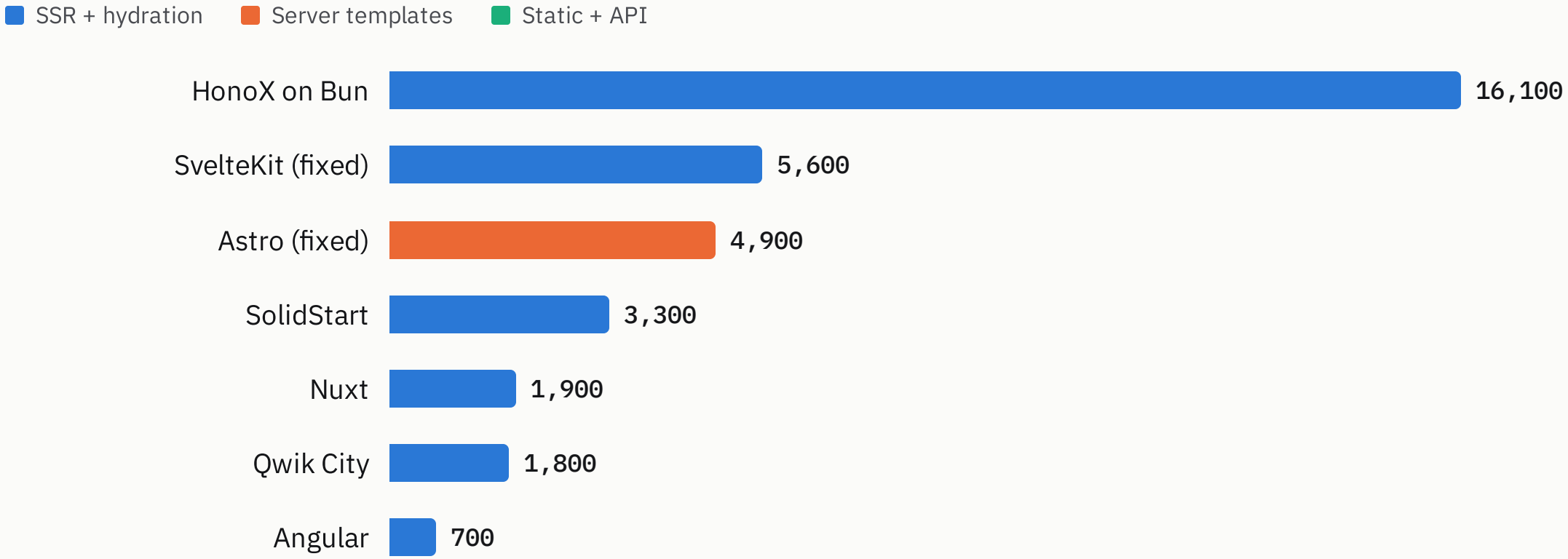
Bun: 1.4× for Next.js, 2.6× for Express



Same code, only the runtime changes. Bun's HTTP server and fetch are native code, so it helps most where the app's own work is small: Express and Hono gain 2.6-2.9×. Next.js gains 1.4× and React Router 4%: React's rendering is the same JavaScript on either runtime. Deno: no gain.

RESULTS · OTHER JS FRAMEWORKS

Compiled string renderers lead



Angular builds a server-side DOM per request. Qwik serializes component state for resumability. Solid and Svelte compile components to string building. SvelteKit and Astro shown after their one-line fix. HonoX (Bun) hydrates only islands: 32× Next.js.

THE BASELINE

Plain Express + EJS on the same Node

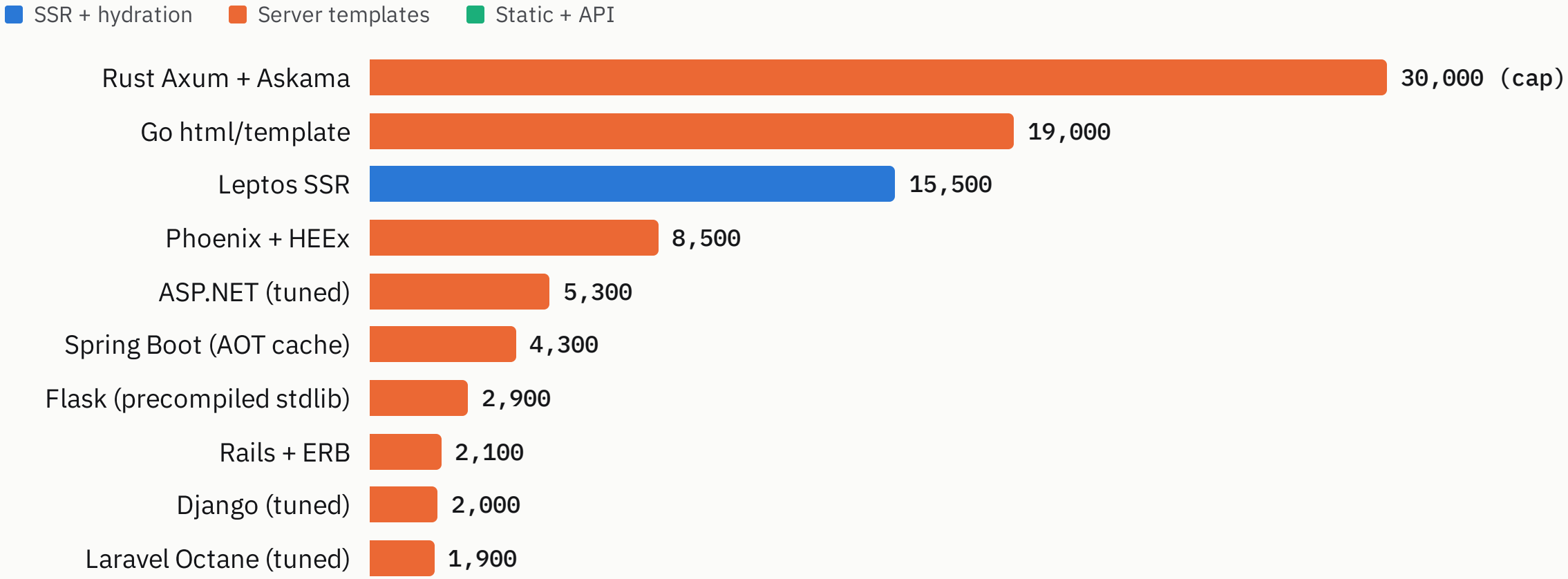
7,900 users held for 5 minutes

16× more than Next.js, same runtime

No component tree, no hydration data, no streaming: a template compiled to a JS function, then string concatenation. Everything a framework spends above this is the framework.

RESULTS · OTHER LANGUAGES

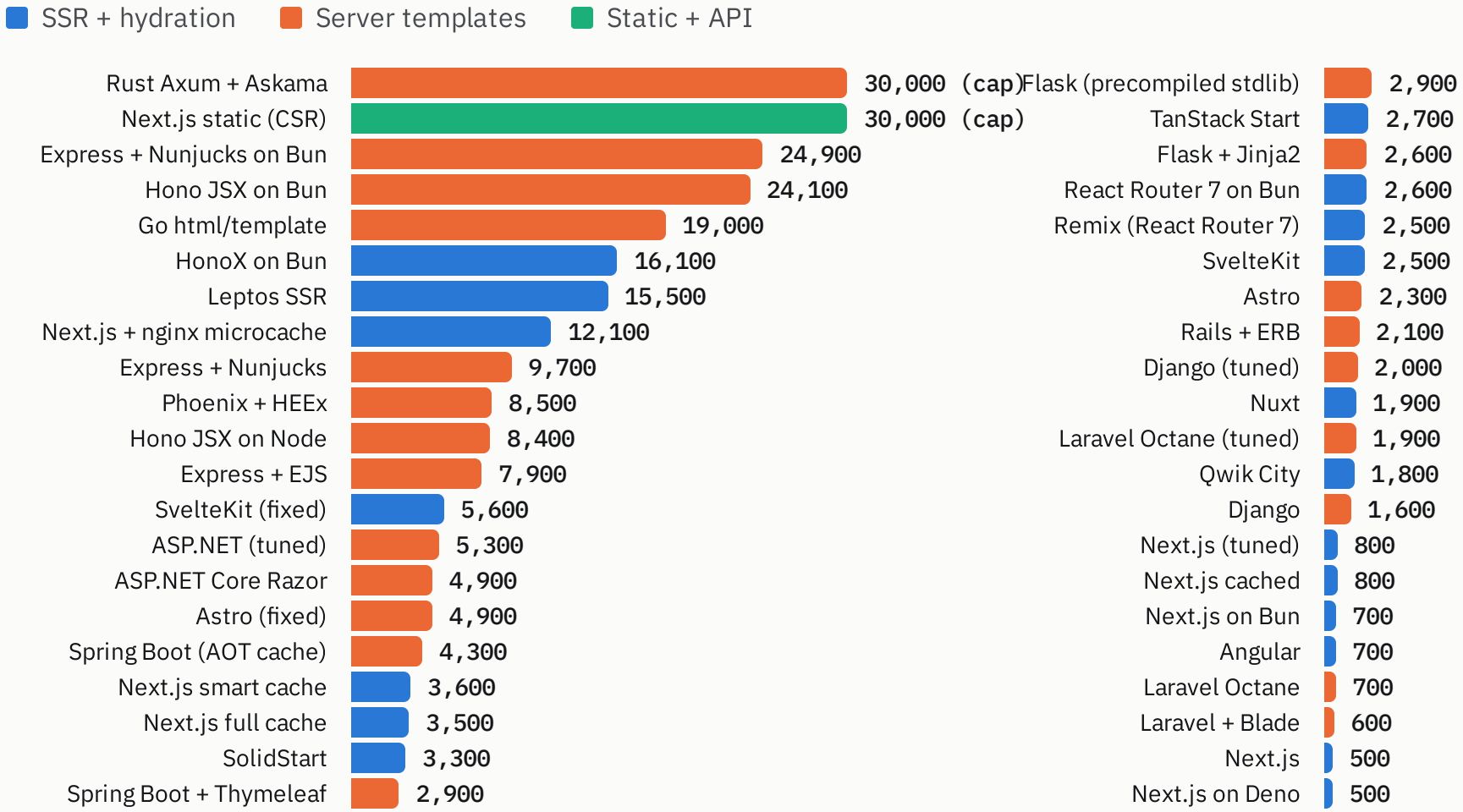
Server templates in other languages



Tuned versions shown. Go failed at 19,500 at 100% CPU. Rust held the 30,000 cap at 54% CPU; its estimated limit is about 50,000. Leptos hydrates like Next.js and still serves 31×.

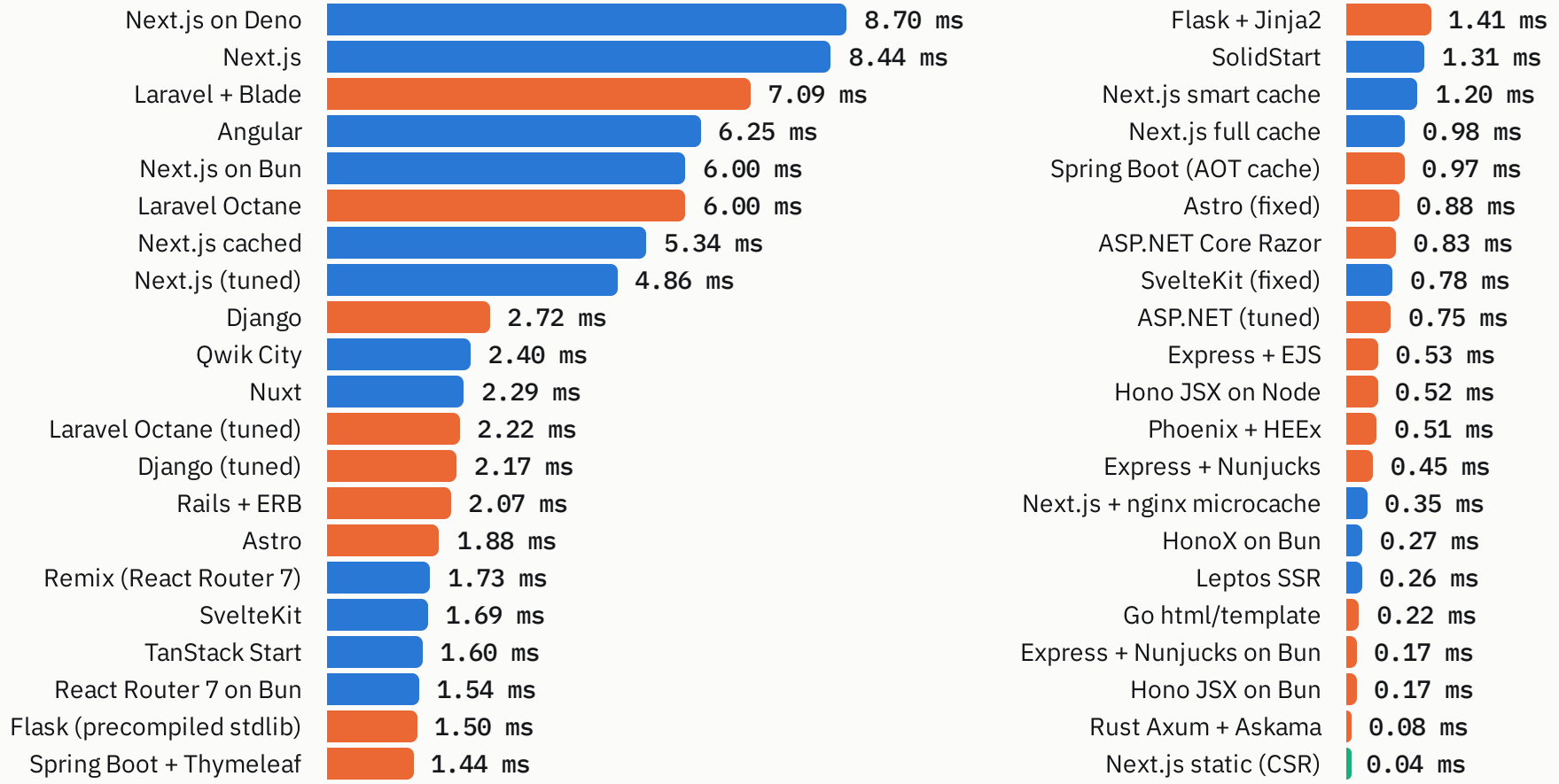
LEADERBOARD

Users held for 5 minutes, 0.4 vCPU



WHY

Every server ran out of CPU, not memory



CPU milliseconds per request at the score. At every server-rendered limit the API used 3-14% of its CPU while the frontend box sat at 95-100%. Score ≈ CPU budget ÷ CPU per request.

What each request pays for

Machinery around React

Next.js: stream plumbing 22%, sockets 12%, GC 11%, actual React rendering ~25%. Plus a ~28 KB inlined RSC payload in a 46 KB page.

A DOM on the server

Angular builds and serializes a document per request, plus hydration annotations and a transfer cache.

State serialization

Qwik writes ~12 KB of component state for resumability; Nuxt, Remix, TanStack and Solid embed 4-7 KB of data.

Strings

Solid, Svelte, EJS, Askama, Razor: compiled templates that append strings. The cheapest way to make HTML.

ODD RESULTS

Profile before you rewrite

STACK	CAUSE	BEFORE	AFTER	GAIN
SvelteKit	date formatter per post card	2,500	5,600	2.2×
Astro	the same formatter	2,300	4,900	2.1×
Laravel + Blade	file sessions, cookies, API connection per request	600	1,900	3.2×
Spring Boot + Thymeleaf	cold JIT → JDK AOT cache	2,900	4,300	1.5×
Django	healthcheck recompiling Python, + Jinja2	1,600	2,000	1.3×
Flask + Jinja2	healthcheck recompiling Python	2,600	2,900	1.1×
ASP.NET Core Razor	idle thread-pool spinning	4,900	5,300	1.1×
Next.js	unused staged/prefetch renders	500	800	1.6×

Each fix is one idiomatic change a team would ship. SvelteKit and Astro built a date formatter for every post card; moving one line more than doubled both.

THE OTHER OPTION

Don't render on the server at all

~42k

users, estimated (30,000 measured: p95 41 ms)

Next.js as a static export behind nginx: the server sends 7 KB shells and JSON, the browser renders. At 30,000 users nginx used 67% of its vCPU and the API 47%.

The cost moves to every visitor's device, first paint waits for JS, and crawlers see an empty shell.

TAKEAWAYS

What to take from this

The renderer is the bottleneck

With a fast API, the HTML renderer decides how many users a box serves: 500 for Next.js, 24,900 for Express on Bun, 30,000+ for Rust.

Profile first

Seven stacks gained 1.1-3× from one fix each: a date formatter, a healthcheck, a JIT warm-up, session files.

For Next.js: cache in front, or change framework

Tuning gives 1.6×, Bun 1.4×. An nginx microcache gives 24× for anonymous pages; React Router or TanStack 5×; server templates 19-60×.

Add CPU, not RAM

Every server-rendered stack hit 95-100% CPU with most of its 2 GB unused.